

EDUCATION PILOT

BY

MUHAMMAD NASIR

*Thesis Submitted to University of Swabi, Khyber Pakhtunkhwa,
in partial fulfillment of the requirement for the degree of*

BACHELOR OF SCIENCE IN COMPUTER SCIENCE



**DEPARTMENT OF COMPUTER SCIENCE
THE UNIVERSITY OF SWABI
KHYBER PAKHTUNKWA, PAKISTAN
SESSION (2021-2025)**

EDUCATION PILOT

By

MUHAMMAD NASIR

*Thesis Submitted to University of Swabi, Khyber Pakhtunkhwa,
in partial fulfillment of the requirement for the degree of*

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

Approved by:

Chairman Supervisory Committee

Dr. Shah Nazir

Co-Supervisor [Field/ Research/ Lab]

Prof. Hammad

Chairman Department of Computer Science

Prof. Mushtaq Ali Shah

**DEPARTMENT OF COMPUTER SCIENCE
THE UNIVERSITY OF SWABI
KHYBER PAKHTUNKWA, PAKISTAN
SESSION (2021-2025)**

EDUCATION PILOT

CERTIFICATE OF SIGNATORY PAGE OF FOREIGN EVALUATORS / EXTERNAL EVALUATORS (PAKISTAN)

External Evaluators / Examiners for Thesis Evaluation

To be approved by the Vice Chancellor

External Examiner-1

To be approved by the Vice Chancellor

External Examiner-2

To be approved by the Vice Chancellor

External Examiner-3

EDUCATION PILOT

Examiners for Thesis Viva Voce

Dr. Shah Nazir

External Examiner

Chairman Supervisory Committee

Prof. Mushtaq Ali Shah

Head Of Department Of Computer Science

**PAGE FOR CERTIFICATE OF TEST REPORT OF PLAGIARISM TO
BE OBTAINED FROM QEC**



SIMILARITY INDEX CERTIFICATE FOR THESIS

Approved Title by ASRB: EDUCATION PILOT

Department: DEPARTMENT OF COMPUTER SCIENCE, THE
UNIVERSITY OF SWABI

Submitted by: MUHAMMAD NASIR

Supervisor: Prof. HAMMAD
DEPARTMENT OF COMPUTER SCIENCE
THE UNIVERSITY OF SWABI

Submission ID: **2753350718**

Similarity Index: **5%**

CHECKED BY Prof. HAMMAD

COUNTER SIGNED BY QEC

Director QEC
University of Swabi, KP

**DIRECTORATE OF QUALITY ENHANCEMENT,
THE UNIVERSITY OF SWABI, KHYBER PAKHTUNKHWA,
PAKISTAN**

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	TABLE OF FIGURES	IX
	TABLE OF IMAGES	X
	ACKNOWLEDGMENTS	XI
	ABSTRACT	12
CHAPTER-1		
1.	INTRODUCTION	13
1.1	PROJECT OVERVIEW/ BACKGROUND	13
1.1.1	PROBLEM BEFORE THE Edu_Pilot	13
1.1.2	PROJECT OBJECTIVES	13
1.1.3	FEATURES OF Edu_Pilot	14
1.2	PROJECT SCOPE	14
1.3	TOOLS	15
1.3.1	SOFTWARE	15
1.3.2	TECH STACK	15
1.3.3	HARDWARE	15
1.4	LIMITATIONS OF THE EDUCATION PILOT	15
1.5	OUTLINE OF THESIS	15
	CHAPTER I- INTRODUCTION	16
	CHAPTER II- REVIEW OF LITERATURE	16
	CHAPTER III- MATERIALS AND METHOD	16
	CHAPTER IV- RESULT AND DISCUSSION	16
	CHAPTER V- CONCLUSION AND RECOMMENDATION	17
CHAPTER-2		
2.	REVIEW OF LITERATURE	18
2.1	ANALYSIS OF EDUCATION PILOT	18
2.2	ANALYSIS	18
2.3	EXISTING SYSTEM ANALYSIS	18
2.4	DRAWBACKS IN THE EXISTING SYSTEM	18
2.5	FEASIBLE STUDY	19
2.5.1	TECHNICAL STUDY	19

2.5.2	FINANCIAL STUDY	19
2.5.3	OPERATIONAL	19
2.5.4	LEGAL STUDY	19
2.5.5	SCHEDULING STUDY	20
2.6	REQUIREMENT	20
2.6.1	FUNCTIONAL REQUIREMENTS	20
2.6.2	NON-FUNCTIONAL REQUIREMENT	20
2.6.3	PERFORMANCE REQUIREMENT	20

CHAPTER-3

3.	MATERIALS AND METHOD	21
3.1	DEFINITION ERD DIAGRAM	21
3.2	IMPORTANCE OF ERD DIAGRAM	21
3.3	ERD DIAGRAM	22
3.4	FLOW CHART DIAGRAM	23
3.5	INTERFACE	23
3.5.1	USER INTERFACE	23
3.5.2	SIDEBAR	24
3.5.3	LOGIN PAGE	25
3.5.4	SIGNUP PAGE	25
3.5.5	HOME PAGE	26
3.5.6	UPLOAD INPUT FORM	26
3.5.7	FILES MENU PAGE	27
3.6	SOFTWARE INTERFACE	27
3.7	HARDWARE INTERFACE	28

CHAPTER-4

4.	RESULT AND DISCUSSION	
4.1	IMPLEMENTATION	30
4.1.1	INTEGRATED DEVELOPED ENVIRONMENT(IDE)	30
4.1.2	BACKEND (MYSQL DATABASE)	30
4.2	CODE SNIPPETS	31
4.3	VISUAL STUDIO CODE	34
4.3.1	INSTALLATION OF VISUAL STUDIO CODE	36
4.4	TESTING	39

4.5 TESTING TECHNIQUES	39
4.5.1 BLACK BOX TESTING	39
4.5.2 WHITE BOX TESTING	40
4.5.3 UNIT TESTING	40
4.5.4 INTEGRATION TESTING	41
4.5.5 SYSTEM TESTING	41
CHAPTER-5	
5. CONCLUSION AND RECOMMENDATION	
5.1 DEPLOYMENT	42
5.2 TARGET AREA FOR THE Edu_Pilot	42
5.3 GEOGRAPHICAL TARGET AREA	43
5.4 CONFIGURATION	43
5.5 DEPLOYMENT PLACES	43
5.5.1 VERCEL DEPLOYMENT	43
5.5.2 NETLIFY DEPLOYMENT	44
5.5.3 CLOUD HOSTING	44
5.6 APPLICATION COSTS	45
5.7 ADVERTISEMENT	46
5.8 HARDWARE INTEGRATION AND FABRICATION	46
5.9 USER TRAINING	46
5.10 FUTURE ENHANCEMENTS	46
LITERATURE CITED	48

TABLE OF FIGURE

FIGURE NO	TITLE	PAGE NO
FIGURE 3.1	ERD DIAGRAM.....	23
FIGURE 3.2	FLOW CHART DIARAM.....	24
FIGURE 3.3	SIDEBAR	25
FIGURE 3.4	LOGIN PAGE.....	26
FIGURE 3.5	SIGNUP PAGE.....	27
FIGURE 3.6	HOME PAGE.....	28
FIGRUE 3.7	UPLOAD FORM.....	29
FIGURE 3.8	SUBSCRIPTION MENU SCREEN.....	29
FIGURE 4.1	PRISMA MODELS CODE.....	31
FIGURE 4.2	THEME PROVIDER CODE	31
FIGURE 4.3	STATE MANAGEMENT CODE.....	32
FIGURE 4.4	LAYOUT.TSX CODE.....	32
FIGURE 4.5	CUSTOM HOOK CODE.....	33
FIGURE 4.6	FORM INPUT CODE.....	33
FIGURE 4.7	BLACK BOX TESTING	40
FIGURE 4.8	WHITE BOX TESTING.....	40

TABLE OF IMAGES

IMAGE NO	TITLE	PAGE NO
	DOWNLOAD AND INSTALL VISUAL STUDIO CODE PROCESS	
IMAGE NO: 4.21		34
IMAGE NO: 4.22		35
IMAGE NO: 4.23		36
IMAGE NO: 4.24		36
IMAGE NO: 4.25		37
IMAGE NO: 4.26		37
IMAGE NO: 4.27		38
IMAGE NO: 4.28		38
IMAGE NO: 4.29		39

ACKNOWLEDGMENTS

In the name of ALLAH, the Most Gracious and Most Merciful, all praises to ALLAH for the Strengths and his blessing to us for the completion of the project. We are very thankful to almighty ALLAH who showers his blessing always on us.

In particular, we are very Thankful to my supervisor Mr. Hammad. He was always helpful and supportive to us all the time till the completion of this project. To be very honest without his guidance, suggestions and encouragement we would not have been to complete this project. He is the source of motivation and guidance for us. We are very thankful for his cooperation and support which was throughout the project. We would like thank for his valuable comments on which basis we achieve our target.

We would like to thank all those who supported us and giving us encouragement for the process of completion of this project. Thanks to my team members colleagues who they supported and encouraged us at the various situations in the completion and successful achievement of this project.

MUHAMMAD NASIR

EDUCATION PILOT

Muhammad Nasir
Department of Computer Science
The University of Swabi,
Khyber Pakhtunkhwa, Pakistan
September 2025

ABSTRACT

EduPilot is an AI-powered web application designed to assist teachers and educational institutions by automating the generation of academic content such as lecture notes, quizzes, assignments, presentations, and examination papers. The system leverages advanced natural language processing models, OCR technology, and intelligent scheduling algorithms to transform raw course materials (uploaded as text, PDFs, or images) into structured, curriculum-aligned teaching resources. By integrating AI with user-friendly interfaces, EduPilot significantly reduces the time and effort educators spend on preparing learning materials, enabling them to focus more on teaching and student engagement. Key features of EduPilot include automatic extraction and cleaning of course content, generation of multi-level quizzes and assessments (easy, medium, complex), creation of midterm and final examinations, and automated scheduling of course timelines with pre-set quiz and exam dates. Teachers can also auto-assign presentation topics, receive AI-generated alternative teaching notes, and optionally share the content with students. The system incorporates real-time notifications to remind students of upcoming assessments and deadlines, thereby fostering better learning discipline. EduPilot adopts a modular architecture, where the backend processes input files, extracts topics, and uses large language models to generate educational content tailored to user requirements. The application ensures accuracy by allowing teachers to review, edit, and save generated content before distribution. Additionally, EduPilot offers a community-driven approach by enabling educators to contribute, customize, and improve shared teaching resources. Beyond classroom use, EduPilot offers institutions useful assistance by standardising the creation of instructional materials, enhancing the effectiveness of course delivery, and lowering administrative burden. EduPilot is a cutting-edge EdTech solution that improves teaching and learning by providing educators with AI-powered resources and advocating for high-quality, scalable, and easily accessible education.

INTRODUCTION

This platform is developed to automate the creation of learning materials like notes, quizzes, assignments, and presentations topics, mid and finalterm papers. EduPilot is a dynamic and intuitive educational web application created to assist teachers, learners, and education institutes. This Web app is a useful tool for improving teaching and learning in the modern educational setting, where quality and effectiveness are equally important. The application uses natural language processing and artificial intelligence to transform course material into curriculum-aligned, structured resources. By enabling teachers to edit and distribute generated content in accordance with classroom requirements, EduPilot fosters a community-driven approach that goes beyond straightforward automation. Different types of quizzes, midterm papers, finalterm papers and generating assignments topics are some of the main features Teachers can also assign presentation topics automatically to ensure equitable student participation. Additionally, EduPilot supports a lot of learning styles and enhances the efficacy and accessibility of education by providing alternate explanations of subjects. EduPilot's AI-powered approach boosts student involvement, lessens teachers' workloads, and enhances academic delivery in general. This Innovative route creates a more organized, structured, productive, and engaging learning environment in addition to saving time.

Project Overview / Background

1.1.1 Problems before the Education Pilot

As we know, Teachers had less time for student engagement before to EduPilot because creating instructional materials like quizzes, notes, midterm, finalterm, presentations and assignments topics was a redundant and repetitive process. Students struggled with unclear or delayed updates regarding assessments, and manual labour frequently resulted in delays, inconsistencies, and additional workload. Lack of automation, scheduling, and intelligent alerts reduced productivity and affect by learning process which make it unorganized. So, By serving an AI-powered platform that expedites content creation, and enhances teaching and learning, EduPilot tackles these problems.

1.1.2 Project Objectives

EduPilot's goal is to give teachers accurate and timely education related resources, such as notes, quizzes, midpaper, finalpaper, and assignments. By using well-structured and easily accessible content, it seeks to improve student learning while lessening the workload of educators. By providing multi-level assessments,

automated scheduling, and customised notifications, this web app also aims to increase efficiency. The ultimate objective is to develop a more intelligent and successful teaching and learning environment.

1.1.3 Features of EduPilot App

Here are some key features of a EduPilot in real-world applications:

- 1. Intelligent Notes Generation:** Converts course material into thorough, structured lecture notes that facilitate instruction and independent study.
- 2. Automated Exam Paper Generation:** Generates the midterm and final term Exam papers, which can save the time of teachers. It also allow teachers to customize the questions.
- 3. Dynamic Quiz Generation:** Creates quizzes, in which a teacher can enter the count of Macq's, True/False, short and long Questions also. which allow teachers to track the learning curve of the students.
- 4. Smart Assignment Topics Creation:** Generates Assignments topics, which is a best way to study the sub topics and gain good understanding on the concepts.
- 5. Presentation Topic Allocation:** Promotes involvement and group learning by automatically assigning students interesting presentation topics.

1.2 Project Scope

The EduPilot project's scope consist of creating an common platform that creates automatically educational materials for instructors, students, and educational institutions. These resources contian notes, quizzzes, presentations, assignments, and exam papers. In order to provide accurate, well-timed, and curriculum-aligned content, it will combine AI models, natural language processing, and user input. The main Features of the this web applications are Generating quizzes, assignments plus presentations topics, mid and finalterm papers via image or text. which will all have an easy-to-use interface. The project will set up ways to measure how well it works, how easy it is to use, and how good the content is. It will be rolled out in stages, using feedback from teachers to make it better. To improve teaching and learning, future updates could include tools like analytics dashboards, personalized learning features, and better connections with learning management systems.

1.3 Tools

1.3.1 Tech Stack:

- TypeScript
- Nextjs

- Langchain
- Python

1.3.1 Software

- Visual studio code
- MySQL database
- Mistral-7b Model

1.3.2 Hardware

- Core i5 8th generation
- Processor 2.50 GH
- RAM 16GB
- Windows 11

1.4 Limitations of the Edu_Pilot

One of EduPilot's limitations is its dependence on artificial intelligence (AI)-generated content, which sometime results in errors or necessitates manual editing by educators to correct the output if there any error. Another drawback of the Edu_Pilot is that a stable internet connection is required for the system, which may limit access in areas with poor connectivity. It uses a lot of computing power so might respond slowly during busy times. There could be concerns about keeping academic data private, secured and safe. Without modification, EduPilot may not fully fit every school's course, and its success depends on teachers being willing to use and adapt content generated by AI. New users might find it hard to learn the system, and some advanced features may only be available in paid versions, which could be a problem for those with limited budgets.

1.6 Outlines of Thesis

The purpose of this section is to provide readers with a concise summary of the thesis. From chapter 1 "Introduction" to chapter 5 "Conclusion," the scheme here delineates the fundamental theme of each chapter.

Chapter I: Introduction

To enhance the teaching and learning process overall, EduPilot is a modern educational application that gives teachers, students and educational institutes access to automatically generated academic resources, such as lecture notes, quizzes, assignments topics, presentations topics, and exam (mid and final term) papers. It minimises the manual process of creating study materials and guaranteeing accuracy and consistency across subjects by giving teachers fast access to well-structured content. This application uses NLP and AI to provide

course-aligned, relevant content that can be modified to meet the needs of various educational levels and classroom settings. The system allows teachers to designate presentation topics, guaranteeing equitable distribution and balanced student involvement. EduPilot helps teachers to save the time, improve student involvement, and hold up better academic performance by blinding automation and flexibility. The app give smarter, faster, and better way for making and sharing study material, so it is very useful for education place like school, college, and training center.

Chapter II: Review of Literature

This chapter is about to express the EduPilot requirements specification, including what the application can do, and how the students and teachers can take advantage of this Application. We will discuss the functional and non-functional requirements of the Application to analyze it's development system. We will also talking about the generating process of quizzes, assignments topics, presentations topics, notes and mid & final term parpers. We will also talking about the performance requirements, such as scalability to accommodate an increasing number of users and institutions, responsiveness of the system, and accuracy of AI-generated content.

Chapter III: Material and method

In this chapter we examine the system design of the Application. System design refers to the process of defining a system's modules, architecture, components, interfaces, and data in accordance with EduPilot's particular requirements. The overall structure of the Application is designed during this phase to guarantee seamless feature integration and operation. The ER Diagram and Flow Chart for system design is made in this Chapter.

Chapter IV: Result and Discussion

In this chapter we will cover the technologies and tools used to develop EduPilot, and also discuss the testing and the deployment procedure. All the stages of the developments are discussed in the implementation section, we also discuss the Programming languages used in the project like Pythoin and Javascript, models like Mistral 7b , IDE's like VsCode and Frameworks like Nextjs and FastAPI. The process of storing the user data with submission data in the database is also discussed here.

Chapter V: Conclusion and Recommendation

In this chapter, we talking about the post-development processes of the Project. After completing the development of a software system, so it needs to be deployed to public his its benefits. We can deploy this on servers, so the users can access the application and can give feedback for improvement. To test that the databases, API's and AI models are working correctly and they are integrated in an accurate way, these are the sub steps in the deployment process. During the deployment phase, Teachers, students and educational institutes

can create quizzes, assignment topics, presentation topics and exam papers. In this chapter we will also talking about the post-deployment phase of the project like, bug fixes, monitoring, and gathering the feedback from users to guarantee that the App is runs smoothly without any inconsistencies.

REVIEW OF LITERATURE

2.1 Analysis of EduPilot App

The main goal of EduPilot is to analyze the needs of the stakeholders such as teachers and students. By talking with teachers and students to know about that what inconsistencies are present in the current education sytem according to course preparation and completion on time. This pre-analysis process is become the foundation of building a better app which will be user friendly and easy to use.

2.2 Analysis

Analysis is the first step in the software development process. In analysis, All the essential tools are checked in the stage. Data is collected from different resources in this stage. Analysis of exam paper generation, assignments topics, presentations topics, quiz requirements, and course content are a few examples of this stage.

2.3 Existing system analysis

Teachers prepare notes, quizzes, assignment topics, presentations topics and exam papers by hand or using the AI like Chatgpt and other AI platforms. This is time consuming process to prepare by hand, on the hand using AI platforms it requires an accurate prompt, and often we cannot provide an accurate prompt. It also affect the accuracy of the questions, sometimes the papers and quizzes are very easy and sometimes it is difficult, so this can hardly affect the students. Sometimes teachers give choice to students that select any topic from the course contents, so it can make conflicts between students because, one topic can be selected by two students. In the current System we only relay on the broad topics and skip the sub topics, that make the deep concepts hide from the students.

2.4 Drawbacks in the Existing Educational System

- **Risk of Human Error:** Preparing quizzes, notes, assignment topics and exam papers manually can result in mistakes. It can affect the accuracy and quality of the educational contents.
- **Inefficiency in Content Creation:** This is time consuming to use the traditional methods and can consume more effort. It can be result in delay in learning material.
- **Cumbersome Documentation:** It is difficult to manage the handwritten documents and files manually.

- **Lack of Automation:** Without smart system, making academic material fast for teacher and student is very hard.

2.5 Feasibility study

“In feasibility study, EduPilot check technical, money, work, and teaching side. It see what system need, how much cost for develop, and if teacher and student will use it or not. It also think about privacy, data use, and how to connect with other platform. This study show what problem can come and what benefit, so project can be possible.”

2.5.1 Technical study

For making correct content, EduPilot technical study look on what technology need, like database, storage, and AI model. It also check if system can grow when more user and data come, and how to join with cloud, learning system, and text extract tool. This study is important to make platform strong and useful.

2.5.2 Financial study

In order to guarantee accurate content generation, EduPilot's technical study focusses on the necessary technologies, including databases, storage solutions, and AI models. It assesses scalability to accommodate expanding users and data, as well as integration with cloud platforms, learning systems, and text extraction tools. The development of a dependable and effective platform depends on this study.

2.5.3 Operational study

EduPilot work like AI helper for education, it use course content to make notes, quiz, assignment, presentation, and exam paper, so learning get better. This operation study look how system help different user like teacher, student, and school. It check main feature, technology, work process, and problem.

2.5.4 Legal study

EduPilot legal study check law and rules that must follow for keep user info and privacy safe. It see how to handle student data and teaching material in right way, and also what happen if content is not correct. This study make sure system follow law and protect user data.

2.5.5 Scheduling study

EduPilot scheduling study focus how system manage time work, so user get content and notice on right time. App auto set quiz, test, assignment, and presentation by course schedule. This study also see timing for content making, deadline, notice, and update, so system work smooth and give good learning help.

2.6 Requirements

Below we write three types of requirements:

2.6.1 Functional Requirements of the Education Pilot

- **Easy Interface:** System must be simple for teacher and student, so they can generate the material easily.
- **Good Accuracy:** EduPilot should make notes, quizzes, and exam paper that are accurate.
- **Flexible Database:** System need to manage small topics and also big subject data.
- **Compatibility:** It must work good with SQL database where submission, and course material are stored.
- **Content Making:** System should create notes, quizzes, assignment topics, presentation topics, and exam papers when user need.

2.6.2 Nonfunctional Requirements of the Education Pilot

- **Good UI:** EduPilot should build in smart way, so it runs smooth for all user.
- **Auto Backup:** System must save backup for note, quiz, and schedule, so no data lost.
- **Many Feature:** Platform give tool like note, quiz, assignment, and presentation for different study need.
- **Future Ready:** Design must allow grow and change for new update and education improve.
- **Data Safe:** EduPilot must keep academic data safe and follow privacy rule.
- **Easy Use:** System must have simple interface, so teacher, student, and admin can use easy.

2.6.3 Performance Requirements of the Education Pilot

In performance need, we check how EduPilot must work for good result. System should handle big data and course content without stop or slow. Server must run all time, so user can always get note, test, and assignment. To ensure that teachers and students receive accurate and timely educational resources, EduPilot should also have fast response times and few loading delays.

MATERIALS AND METHOD

The process of defining a system's modules, architecture, components, interfaces, and data flow in accordance with particular requirements is known as system design.

3.1 ER Diagram for "EduPilot"

EduPilot's Entity-Relationship (ER) Diagram lists the data entities, their connections, and each entity's attributes. In order to ensure smooth creation and distribution of educational content, the app usually manages courses, topics, quizzes, assignments, exam papers, students, and teachers.

3.1.1 Importance of ER Diagram for EduPilot

The importance of EduPilot extends beyond simplifying teaching tasks. It enhances **learning efficiency, reduces content preparation time, supports academic scheduling, and improves student engagement**. Teachers, students, and academic administrators benefit from a centralized platform to manage course materials, assessments, and presentations. By organizing data effectively and supporting automated content generation, EduPilot contributes to a smarter educational workflow, allowing educators to focus on teaching while ensuring students have timely access to study resources.

3.2 Tables and Attributes

1. *Submissions*

- **Attributes:**
 - `id`: Primary key, auto-incremented integer.
 - `title`: Text field, auto-generated by Mistral-7b Model.
 - `status`: Text field (varchar), represents the status of the submission.
 - `userId`: Foreign key, links to `user` table.

2. *Users*

- **Attributes:**
 - `id`: Primary key, auto-incremented integer.
 - `name`: Text field (varchar), represents the sector name.
 - `email`: Text field (varchar), should be unique.
 - `password`: Text field (varchar), represents the password of the user.

Relationships

1. **users** has a **one-to-many** relationship with **submissions** (multiple submissions can belong to one user).

Keys

- **Primary Keys:** `id` in each table uniquely identifies each record.
- **Foreign Keys:**
 - `userId` in **submissions** links to `id` in **users**

3.1 ERD DIAGRAM

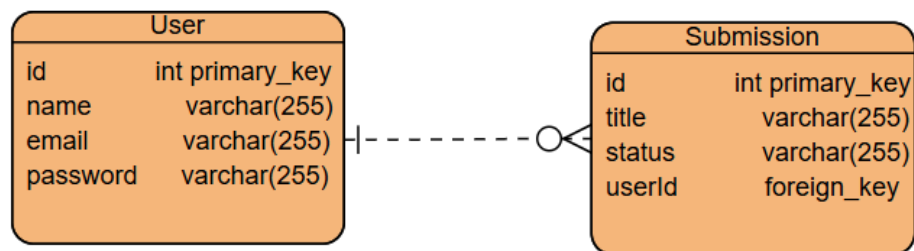


FIGURE 3.1: ERD DIAGRAM

FLOW CHART DIAGRAM

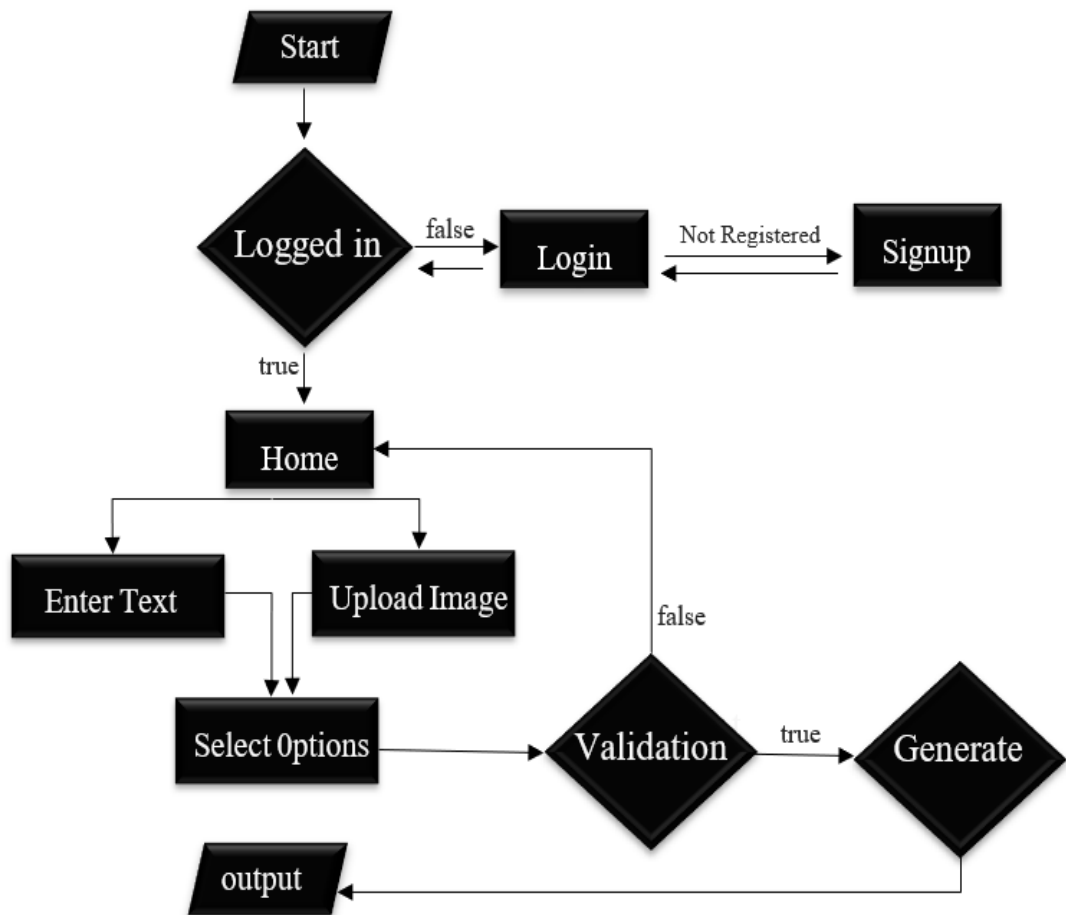


FIGURE 3.2: FLOW CHART DIAGRAM

3.5 Interface

There are some types of interfaces which are given bellow:

3.5.1 User interface

In this section, we study the user interface, focusing on making it easy and intuitive for teachers and students. A graphical interface is preferred over a command-line interface because it simplifies interactions and improves usability. EduPilot will use interactive components such as buttons for generating notes, quizzes, assignments, and presentations, navigation menus, validation alerts, and progress indicators. Users can simply click or tap to perform tasks, ensuring a smooth and efficient experience without technical difficulties.

3.5.2 Sidebar

The Sidebar is a component used to show multiple components. Logo , Close Sidebar Button, New Chat Button, Title of contents you created earlier, Dark/Light mode toggle Button.

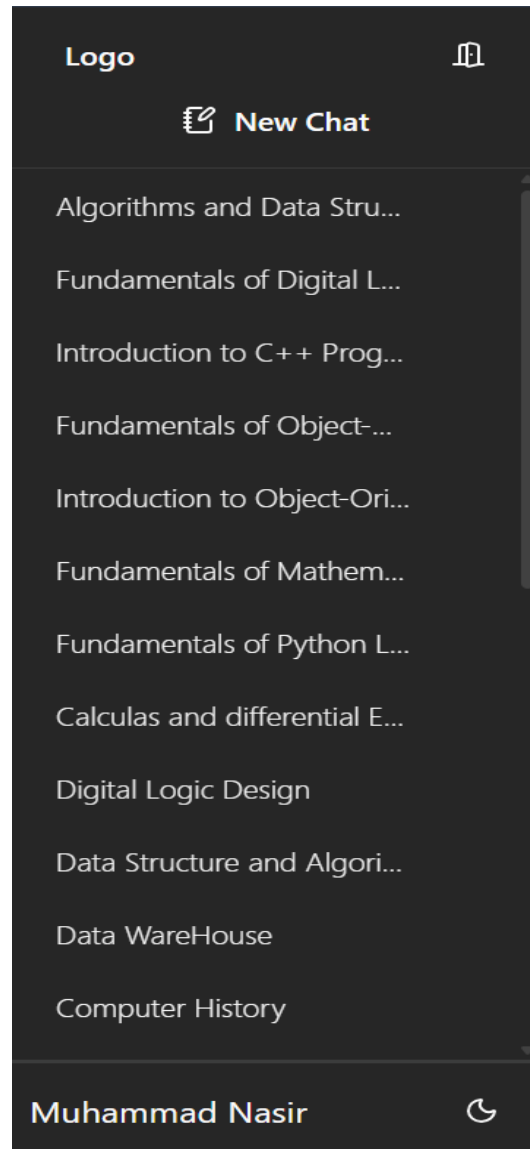


FIGURE 3.3: Sidebar Screen

3.5.3 Login Screen

This is the login page of the EduPilot, used to authenticate the user, when a user wants to get access to the Website.

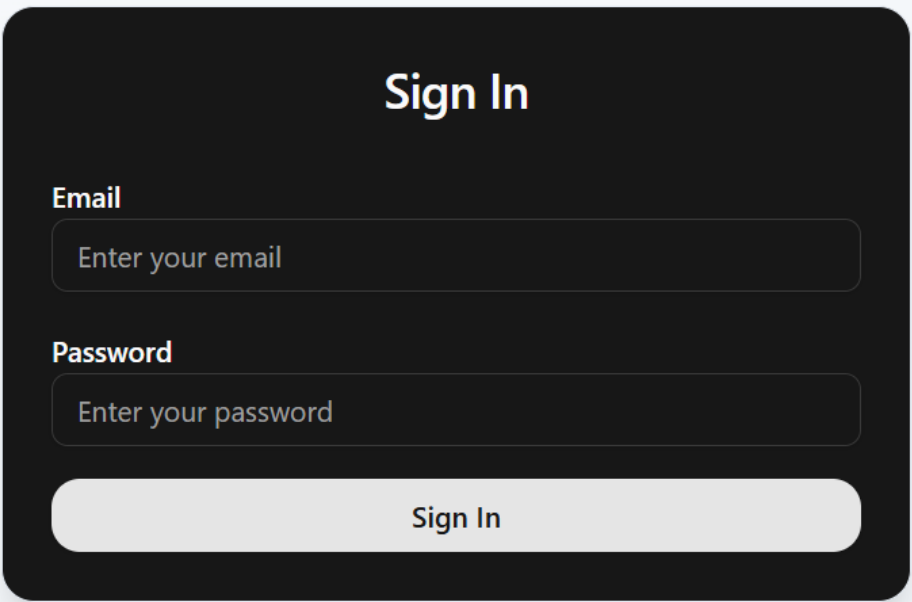
The login screen features a dark-themed card with rounded corners. At the top, the text "Sign In" is displayed in a large, white, sans-serif font. Below this, there are two input fields. The first is labeled "Email" in a smaller white font, with a placeholder text "Enter your email" inside a dark rounded rectangle. The second is labeled "Password" in a smaller white font, with a placeholder text "Enter your password" inside a dark rounded rectangle. At the bottom of the card is a wide, light-colored button with rounded ends, containing the text "Sign In" in a dark font.

FIGURE 3.4: LOGIN SCREEN

3.5.4 Signup Screen

This is the signup screen of the EduPilot, used to allow user to register it self to the website, and when the user want to login, It will be authenticated using the credentials provided by the user and saved in the database.

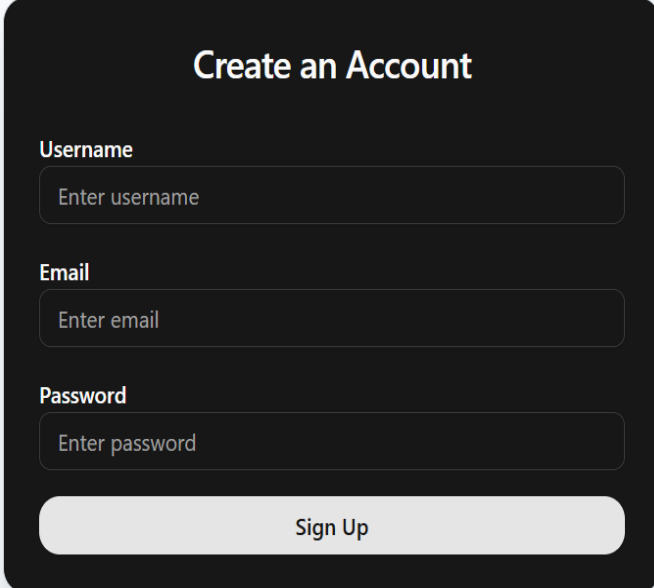
The signup screen features a dark-themed card with rounded corners. At the top, the text "Create an Account" is displayed in a large, white, sans-serif font. Below this, there are three input fields. The first is labeled "Username" in a smaller white font, with a placeholder text "Enter username" inside a dark rounded rectangle. The second is labeled "Email" in a smaller white font, with a placeholder text "Enter email" inside a dark rounded rectangle. The third is labeled "Password" in a smaller white font, with a placeholder text "Enter password" inside a dark rounded rectangle. At the bottom of the card is a wide, light-colored button with rounded ends, containing the text "Sign Up" in a dark font.

FIGURE 3.5: SIGNUP SCREEN

3.5.5 Home Screen

The Home page of the Education Pilot, consists of two main components: Sidebar and Upload Form Component.

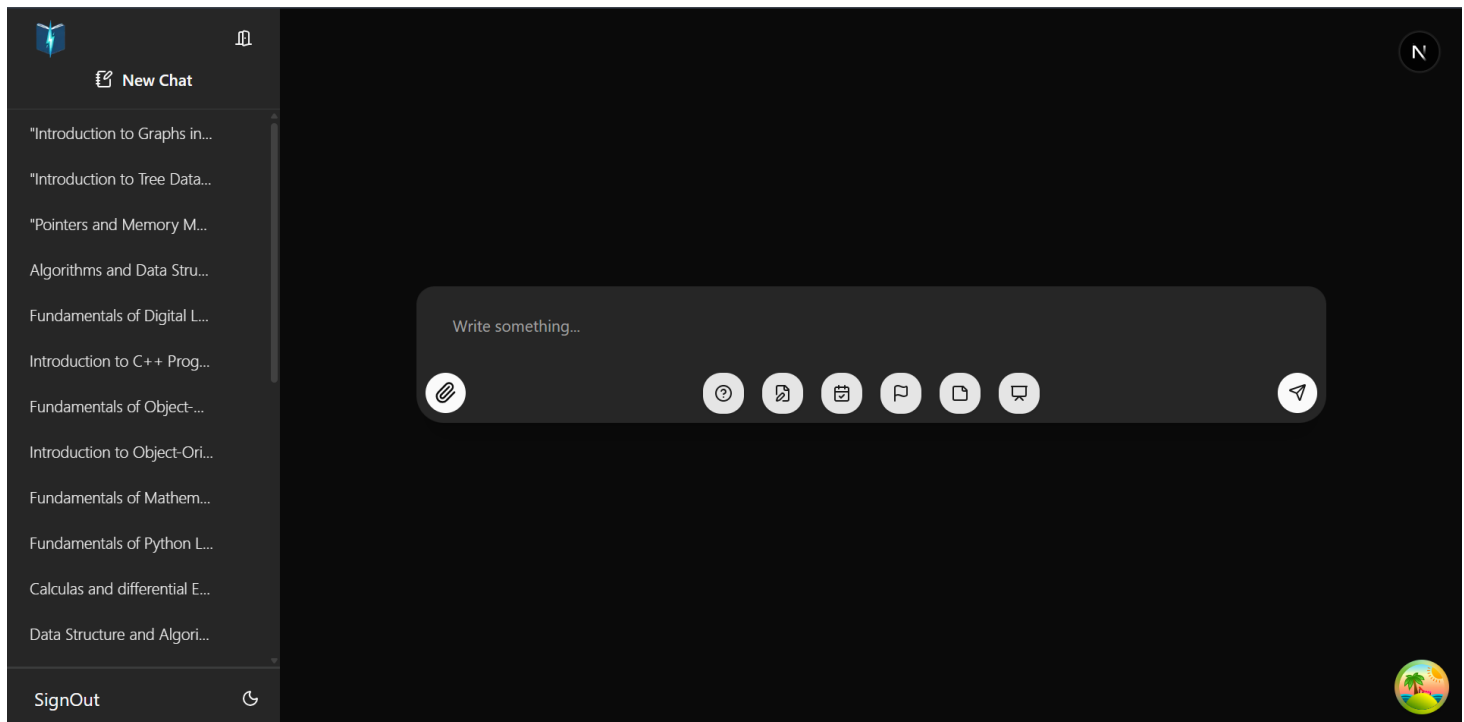


FIGURE 3.6: HOME SCREEN

3.5.6 Upload Image/Text Form

The Upload Form component is used to take either image or text from the user and select options.



FIGURE 3.7: UPLOAD FORM COMPONENT

3.5.7 Files Menu

Renders all the files generated by the request of a user, When we click on a Submission title.

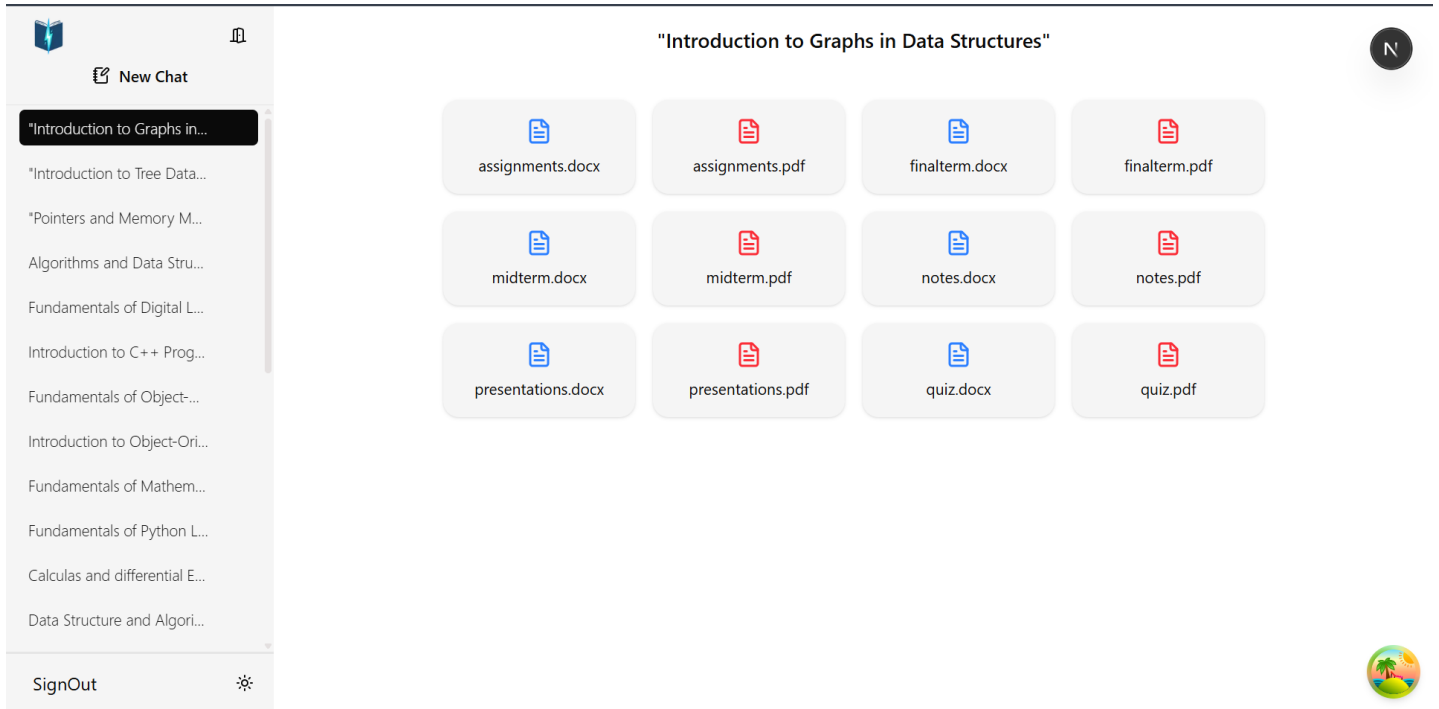


FIGURE 3.8: SUBMISSION FILES MENU

3.6 Software Interface

The software interface describes the environment, tools, and technologies used for developing and running the **EduPilot** application. Being a **web-based system**, EduPilot runs entirely inside modern web browsers and does not require a specific operating system for end users.

1. Browser Environment (for End Users)

- EduPilot runs smoothly on all modern web browsers such as **Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari**.
- No installation is required; users only need internet connectivity and a browser.

2. Development Environment

- The project is developed primarily using **Visual Studio Code**, which supports debugging, version control, and extensions for JavaScript/TypeScript.

3. Frontend Framework

- **Next.js (React-based framework)** is used for building the user interface with **TypeScript** for type safety and scalability.
- The frontend ensures responsive design, interactive UI, and smooth navigation.

4. Backend and AI Integration

- The backend APIs are developed using **Python (FastAPI/Flask)** for handling AI requests.
- **LangChain** is integrated to orchestrate prompts and manage the flow between the user input and AI model.
- **Mistral Model** (open-source LLM) is used for text generation, content creation, and intelligent responses.

5. Database Server

- **MySQL** is used to store user data, course submissions, generated notes, and related academic content.
- **Prisma ORM** is used for database queries and schema management.

6. Dependency & Package Management

- **npm** and **yarn** are used for managing JavaScript/TypeScript dependencies.
- **pip** (Python package manager) is used for installing AI and backend dependencies.

7. Version Control & Collaboration

- **Git/GitHub** is used for source code management, versioning, and collaboration among developers.

3.7 Hardware Interface

EduPilot is a **web-based application** that does not require installation on end-user devices. The hardware requirements are minimal, as users can access the system through any modern web browser. However, both **end-users** and the **server environment** need to meet certain requirements for smooth operation.

For End-Users (Students & Teachers):

- **Device Type:** Smartphone, Tablet, Laptop, or Desktop.
- **RAM:** Minimum **2 GB RAM** (4 GB or higher recommended for smooth browsing).
- **Processor:** Dual-core processor or higher.
- **Storage:** Only requires space for browser cache (~100 MB free space).
- **Operating System:** Any OS supporting modern browsers (Windows, macOS, Linux, Android, iOS).
- **Internet Connection:** Stable broadband/mobile data connection (minimum **2 Mbps** recommended).
- **Browser Compatibility:** Google Chrome, Mozilla Firefox, Safari, or Microsoft Edge (latest versions).

For Server (Deployment Environment):

- **Processor:** Quad-core or higher (Intel/AMD).
- **RAM:** Minimum **8 GB RAM** (16 GB recommended for AI workloads).
- **Storage:** At least **50 GB SSD** for application files, database, and generated content.
- **Server Environment:** Linux-based server (Ubuntu recommended) with Node.js, Python, and MySQL installed.
- **Cloud Compatibility:** Can be deployed on platforms such as **Vercel, AWS, Google Cloud, or Azure.**

RESULT AND DISCUSSION

4.1 IMPLEMENTATION

In this chapter, we discuss the tools and technologies used in developing EduPilot, such as IDEs, emulators, database connections with MySQL, and the backend structure. The chapter also covers the modules for both teachers and students, explaining how each interacts with the system. Implementation represents the output of the project idea, developed using specific tools to generate notes, quizzes, assignments, presentations, and exam papers efficiently.

4.1.1 Integrated Developed Environment (IDE)

An Integrated Development Environment is a software application that provides complete facilities to developers for creating EduPilot. An IDE generally includes a source code editor, build automation tools, and a debugger to test the code flow. It helps programmers write software code efficiently by combining features such as editing, building, testing, and packaging into one platform. IDEs improve developer productivity, making the process of coding EduPilot easier, similar to how writers use text editors or accountants use spreadsheets.

4.1.2 Backend (MY SQL Database)

We are used the firebase (My SQL database is a Backend-as-a-Service (BaaS) app development platform that provides hosted backend services such as a Realtime database, cloud storage, authentication, crash reporting, machine learning, remote configuration, and hosting for your static files.).

My SQL provide the following services:

- **Data Storage:** MySQL stores data in tables and organizes it for easy access and management.
- **SQL Queries:** MySQL allows you to use SQL to retrieve, update, and manage data in the database.
- **Security:** MySQL provides features like user authentication and access control to secure your data.
- **Backup and Restore:** If system fail, MySQL give tool to save and bring back data.
- **Replication:** To increase performance and dependability, MySQL can replicate data across several servers.
- **Always Available:** MySQL have feature so database stay online even if problem come.
- **Can Grow:** When app get big, MySQL can handle very large data.

- **ACID Rule:** MySQL follow ACID, so transaction stay safe and trustable.
- **Work on Many System:** MySQL run on Linux, Mac, and Windows.
- **Cloud Use:** MySQL can use in cloud with managed service.
- **Stored Procedure:** In MySQL, you can write SQL code one time and use again for process.
- **Keep Data Correct:** MySQL use rule and limit to keep data right.
- **Open Source:** MySQL is free, but Oracle give paid support.

4.2 Code Snippets:

```
You, yesterday | 1 author (You)
model User {
  id      Int           @id @default(autoincrement())
  username String
  password String
  email   String        @unique
  submissions Submission[]

  @@map("users") // 🖱️ force DB table name
}

You, yesterday | 1 author (You)
model Submission {
  id      Int           @id @default(autoincrement())
  title   String
  status  String        @default("pending")
  userId  Int
  user    User          @relation(fields: [userId], references: [id])

  @@map("submissions") // 🖱️ force DB table name
}
```

Figure 4.1: Prisma Models

```
1  "use client"
2
3  import * as React from "react"
4  import { ThemeProvider as NextThemesProvider } from "next-themes"
5
6  export function ThemeProvider({
7    children,
8    ...props
9  }: React.ComponentProps<typeof NextThemesProvider>) {
10    return <NextThemesProvider {...props}>{children}</NextThemesProvider>
11  }
```

Figure 4.2: Theme Provider Code

```

import { create } from "zustand";

...

interface SidebarState {
  selected: string | null;
  title: string | null;
  setSelected: (id: string | null) => void;
  setTitle: (name:string | null) => void;
}

const useSidebarStore = create<SidebarState>((set) => ({
  selected: null,
  title: null,
  setTitle: (name) => set({title:name}),
  setSelected: (id) => set({ selected: id }),
}));

export default useSidebarStore;

```

Figure 4.3: State Management Code

```

"use client"

import * as React from "react"
import { ThemeProvider as NextThemesProvider } from "next-themes"

export function ThemeProvider({
  children,
  ...props
}: React.ComponentProps<typeof NextThemesProvider>) {
  return <NextThemesProvider {...props}>{children}</NextThemesProvider>
}

```

Figure 4.4: Layout.tsx Code

```

import { UploadFormSchemaType } from "@lib/schemas";
// Axios
import axios from "axios";
// Helper function
import buildFormData from "@app/helpers/BuildFormData";

const usePostSubmission = () => {
  const addSubmission = useMutation({
    mutationFn: (newSubmission: UploadFormSchemaType) => {
      const formData = buildFormData(newSubmission);
      return axios.post("/api/submission/post", formData);
    },
    onSuccess: (res) => {
      console.log("Submission sent Successfully!", res.data);
    },
    onError: (err) => {
      console.log("Some Error Occurred!", err.message);
    },
  });
  return addSubmission;
};

export default usePostSubmission;

```

Figure 4.5: Custom Hook Code

```

import { Input } from "@components/ui/input";
// Custom type any
import { CustomType } from "@lib/types";

export default function UploadPrompt({ control, isImage }: { control: CustomType, isImage: boolean }) {
  return (
    <FormField>
      control={control}
      name="prompt"
      render={({ field }) => (
        <FormItem className="flex-1 px-2">
          <FormControl>
            <Input
              readOnly={isImage}
              {...field}
              value={field.value ?? ""}
              type="text"
              placeholder={` ${isImage? "" : "Write something..."} `}
              className={`border-none outline-none bg-secondary shadow-none focus:ring-0 focus:border-transparent`}
            />
          </FormControl>
        </FormItem>
      )}
    </FormField>
  );
}

```

Figure 4.6: Form Input Code

4.3 Visual Studio Code

Microsoft make Visual Studio Code (VS Code), it is free and open-source editor for write and debug modern web and cloud app. It support many language like Python, C++, JavaScript and more, by own feature and extra extension. Syntax highlighting, IntelliSense (autocomplete), version control integration, debugging tools, and a robust ecosystem of extensions to boost productivity are just a few of the features that make Visual Studio Code a lightweight yet potent environment. Because of its high degree of customisation and compatibility with Windows, mac OS, and Linux, it is a preferred option for developers in a variety of sectors.



IMAGE NO:4.21

- **Size:** 200-300 MB
- **License:** Freeware, MIT License
- **Operating system:** Windows, mac OS, Linux
- **Stable release:** 1.84.0 (October 2024)
- **Programming languages:** JavaScript, Type Script, Python, C++, Java, Go, HTML, CSS, and more
- **Developer:** Microsoft, Open Source Community
- **Original author:** Microsoft

4.3.1 Installation of Visual Studio Code (System Requirements)

Before downloading and installing Visual Studio Code, the following system requirements are essential.

- Microsoft Windows 7/8/10/11 (32-bit or 64-bit), mac OS 10.11 or later, Linux (various distributions supported).
- Minimum 1 GB RAM (2 GB or more recommended for larger projects).

- Minimum 200 MB of free disk space for installation; 1 GB or more recommended for extensions and additional tools.
- 1.6 GHz or faster processor (multi-core recommended for better performance).
- 1024x768 (higher resolution recommended for better experience).
- For specific language support (e.g., Python, Java, and C++), respective runtimes or SDKs may need to be installed.

Download and Install Visual Studio Code

Step 1

To download the Visual Studio Code, visit the official Visual Studio Code website in your web browser.

<https://code.visualstudio.com/download>

Step 2

Click on the window option

Step 3

Click on “download” button

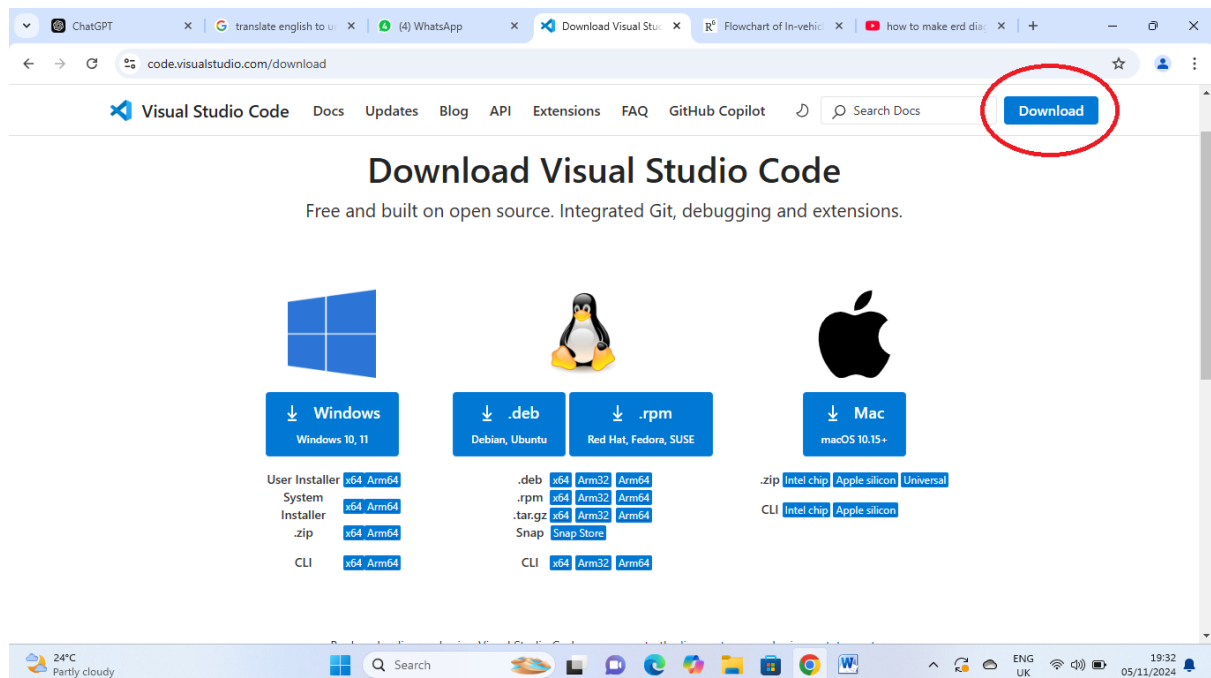


IMAGE NO: 4.22

Step 4: When the downloading process is complete then install the “VS code”

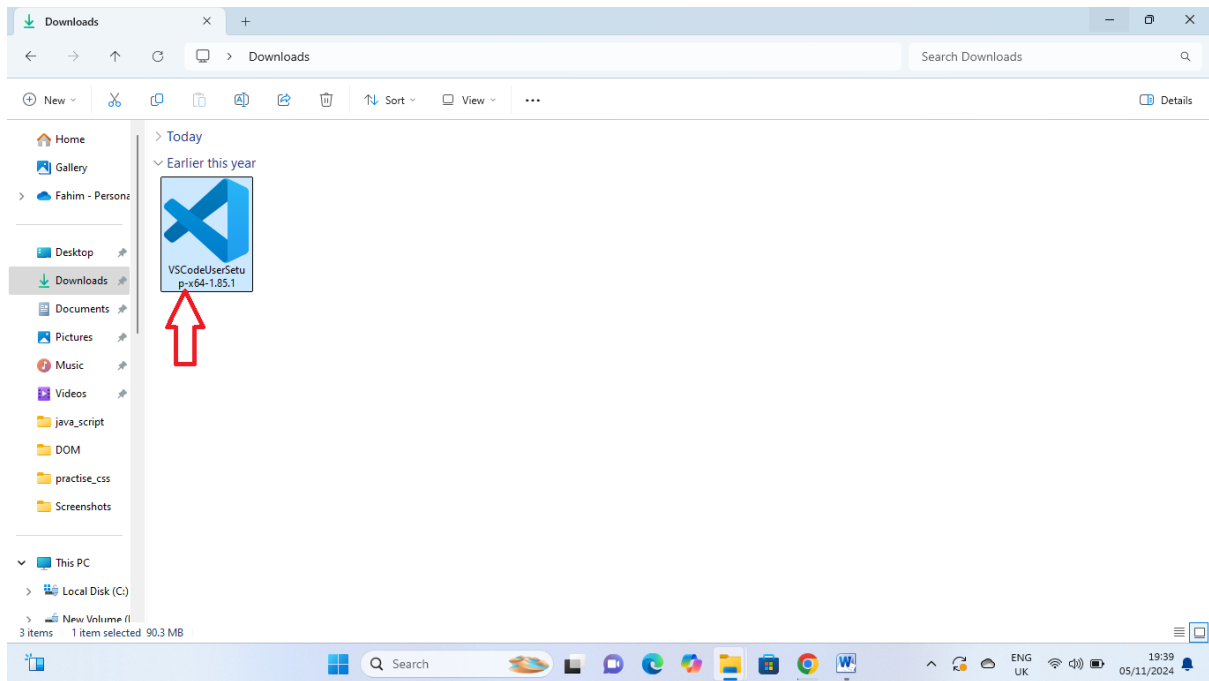


IMAGE NO: 4.23

Step 5

Click on “I accept the agreement”

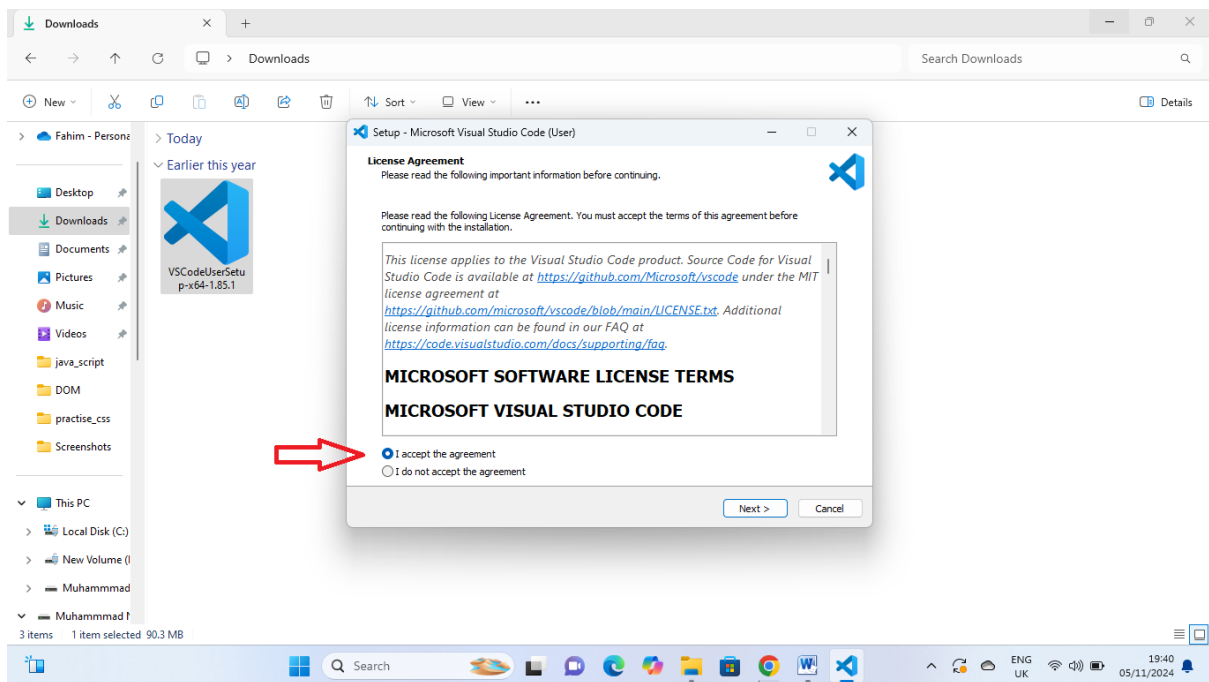


IMAGE NO 4.24

Step 4: Click on the “next” option

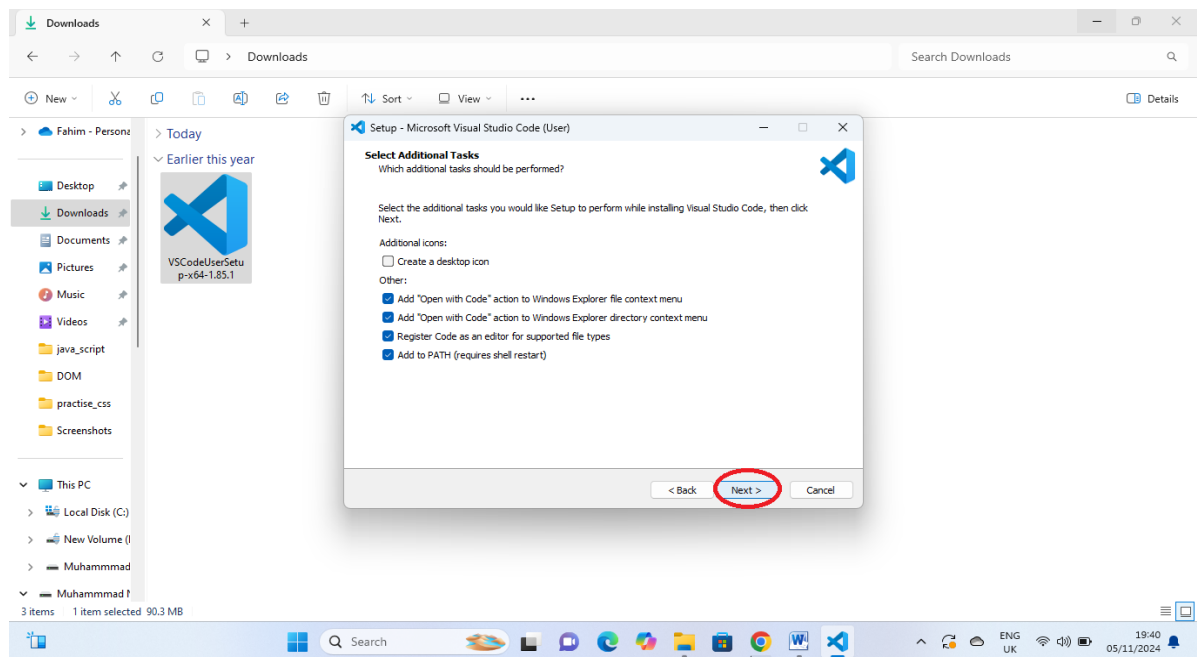


IMAGE NO:4.25

Step 7:
Click on “install” option

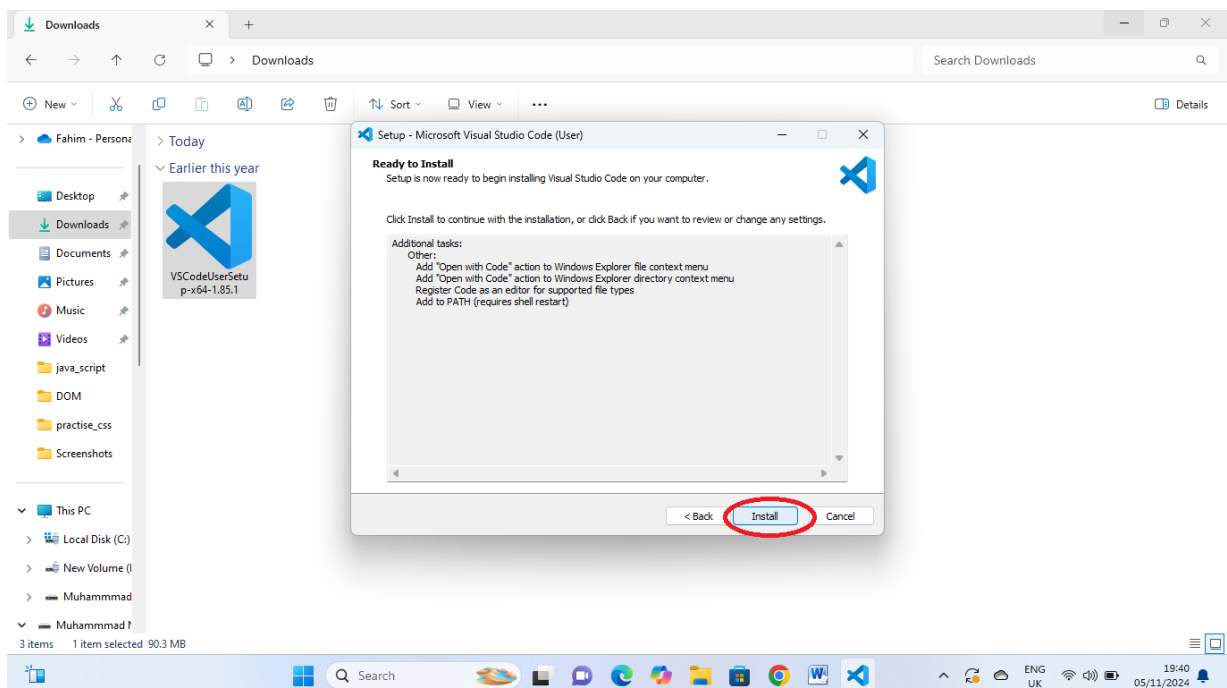


IMAGE NO: 4.26

Step 8:

“Downloading” is started wait until the downloading is complete

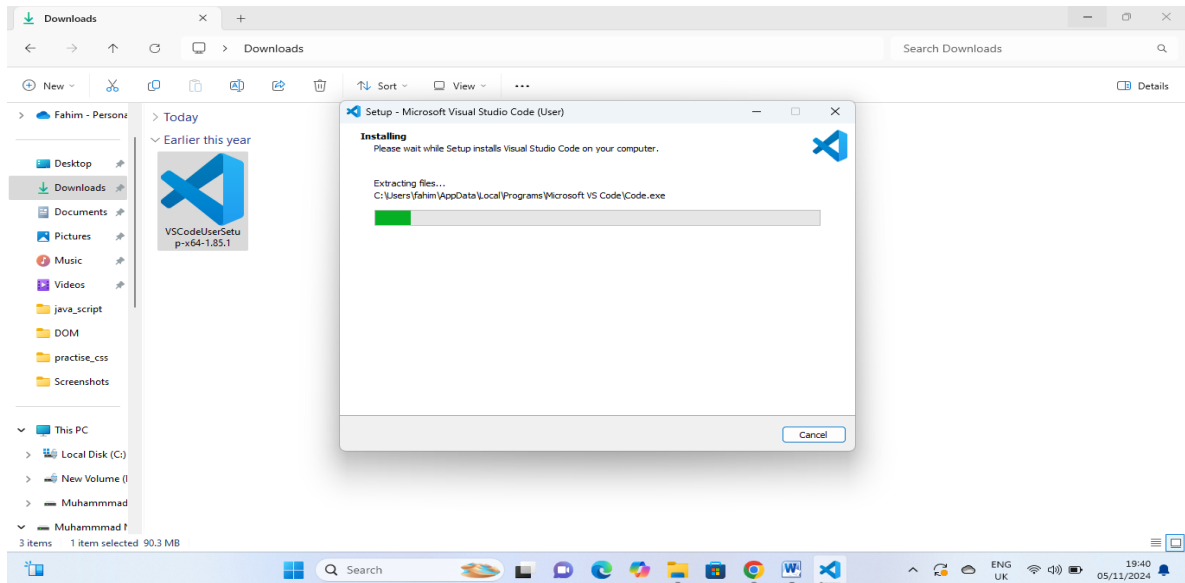


IMAGE NO 4.27

Step 9: When the installing is complete, then press the “Finish” button

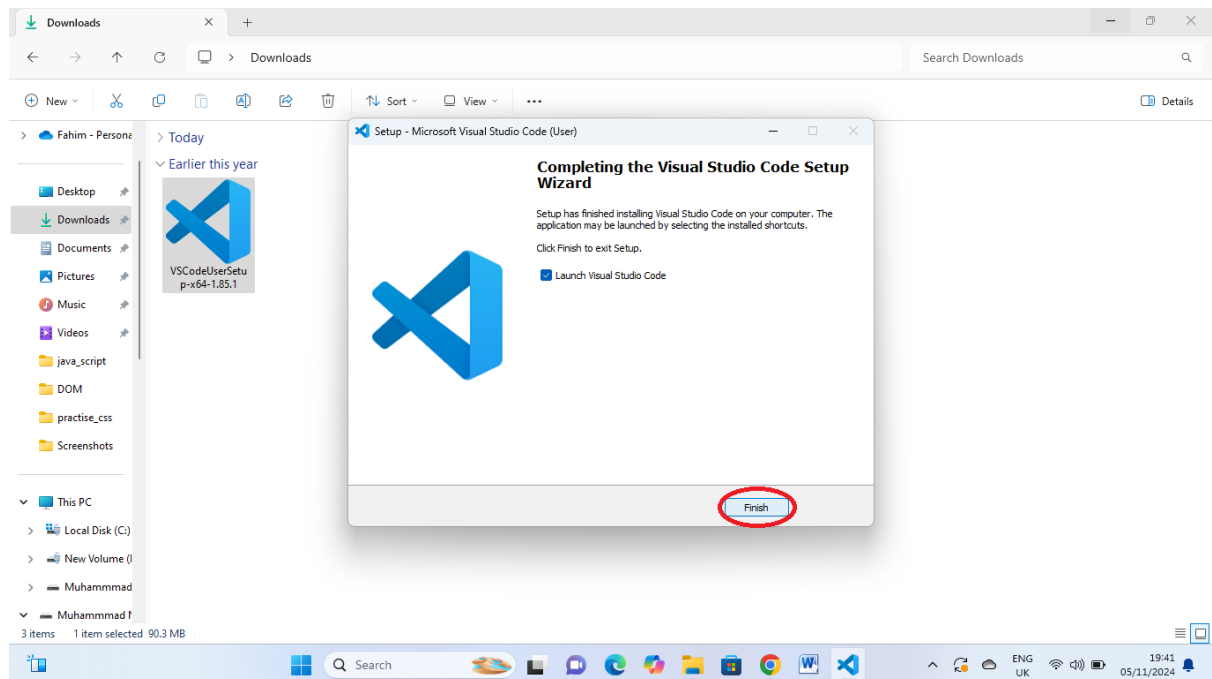


IMAGE NO: 4.28

Step 10:

Open the “Visual Studio Code” and open the folder and create the PHP new file

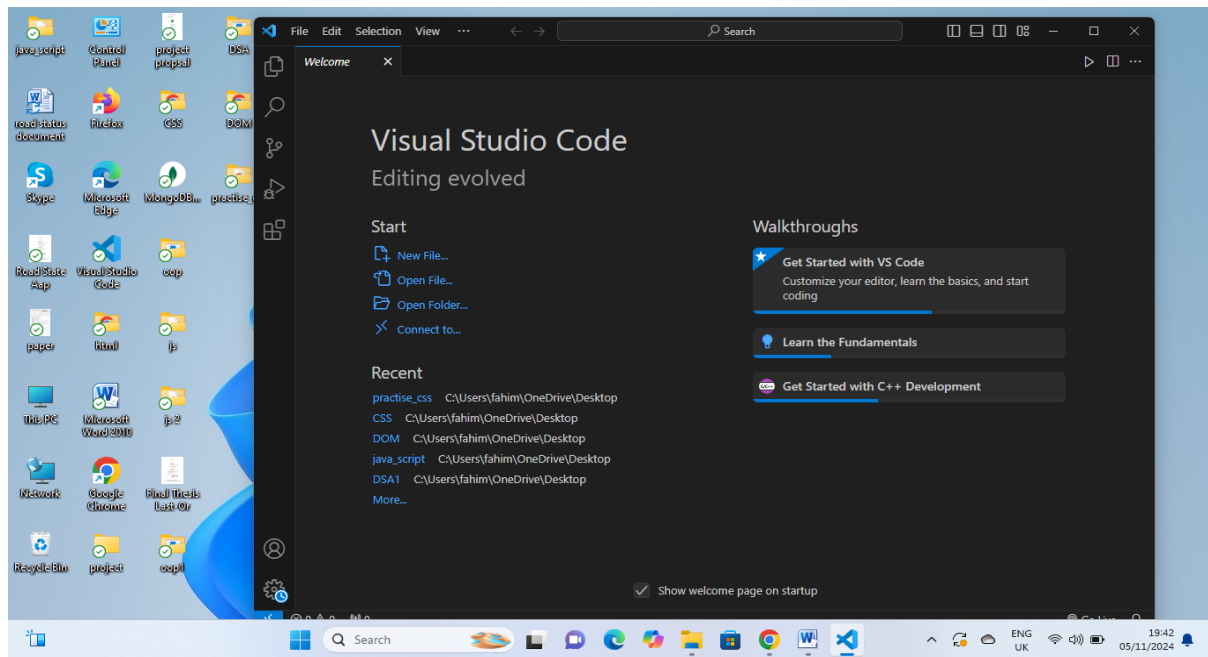


IMAGE NO: 4.29

4.4 Testing

Testing techniques are methods used to ensure that EduPilot meets its functional and performance requirements. These methods aid in finding bugs, improving usability, and guaranteeing seamless network and device operation. Common methodologies include Unit Testing (analysing individual modules, like quiz generation), Integration Testing (making sure modules, like notes, assignments, and scheduling, work together), Black Box Testing (confirming functionality without viewing internal code), White Box Testing (examining internal code structure), and System Testing (evaluating the entire EduPilot system end-to-end).

4.5 Techniques of Testing

4.5.1 Black Box Testing

- This method confirms EduPilot's operation without necessitating an understanding of the internal structure or code.
- It makes sure the system acts appropriately from the user's point of view by concentrating on inputs and expected outputs.
- Input/output management, user interface testing, and application flow testing are a few examples.

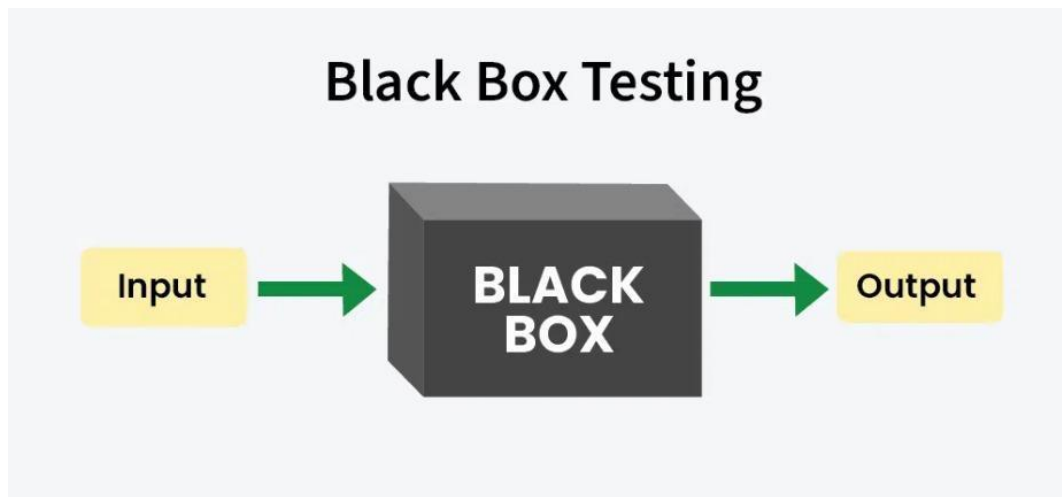


Figure 4.7: Black Box Testing

4.5.2 White Box Testing

- Need to know inside code and structure of EduPilot for this method.
- It check each module, like quiz making and scheduling, by test code path, logic, and data flow to see if work right.
- White box testing use way like path test, decision test, and statement test, and mostly developer do it.

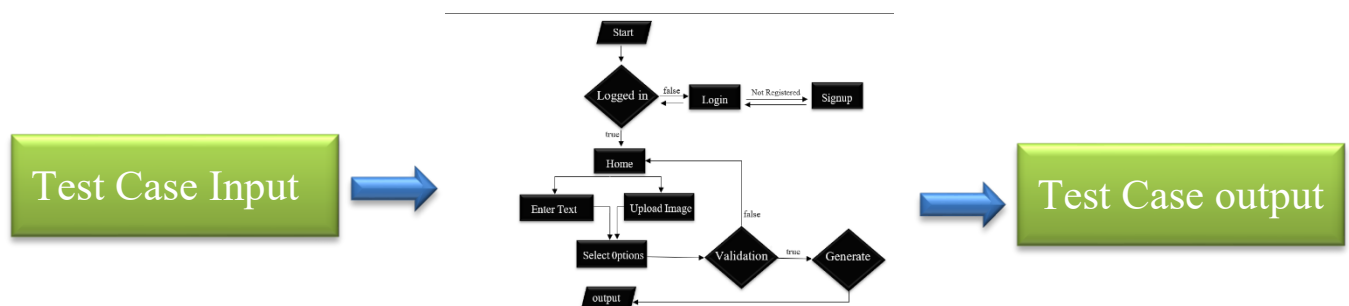


Figure 4.8: White Box Testing

4.5.3 Unit Testing

- In Unit Testing, individual components or functions of EduPilot are tested separately to verify their correctness.
- It is usually carried out by developers and guarantees that, prior to integration with other modules, each "unit," such as note generation, quiz creation, or assignment scheduling, functions independently.
- Most unit tests are automated, allowing quick re-testing whenever the code is modified or updated.

4.5.4 Integration Testing

- This technique tests how different modules or components of EduPilot work together as a complete system.
- It verifies that the interactions between modules (e.g., backend APIs, database storage, quiz/assignment generation, and frontend display) function smoothly and as expected.
- Integration testing helps identify issues that arise when combining modules, ensuring seamless cooperation across the EduPilot platform for teachers and students.

4.5.5 System Testing

- System testing is a high-level test of the entire EduPilot platform as a complete system.
- It validates that all modules (content extraction, note generation, quiz/assignment scheduling, and course timeline management) work together as expected in a real-world teaching and learning environment
- System testing ensures that EduPilot meets its functional, performance, usability, and security requirements, providing a smooth and reliable experience for teachers and students.

CONCLUSION AND RECOMMENDATION

5.1 What is deployment?

The process of delivering the finished system to the client or end users for real use comes after the software or an application has been developed. To deploy a website, testing is essential, In testing we ensure that the app is running properly on the intended setting, we have preprocesses before the deployment of a website. After the completion of all the pre-procedures, we deploy the website on platforms like Amazon AWS, Google Cloud, Vercel and Netlify etc.

5.2 Target Area for the EduPilot:

1. Teachers and Educators:

- Individuals responsible for creating and managing course content, quizzes, assignments, and exams.
- **Age group:** 25–60 years.
- **Devices:** Laptops
- **Use case:** They use EduPilot to generate notes, quizzes, assignments, and presentation topics efficiently, saving time and improving teaching productivity.

2. Students:

- Learners enrolled in schools, colleges, or universities.
- **Age group:** 15–30 years.
- **Use case:** They rely on EduPilot to access study materials, practice quizzes, and track assignments or exam schedules.

3. Academic Institutions:

- Schools, colleges, and universities managing large numbers of courses and students.
- **Use case:** EduPilot helps institutions standardize content creation, schedule assessments, and manage academic resources efficiently.

4. Tutors and Private Trainers:

- Individuals providing personalized or online tutoring.
- **Use case:** They use EduPilot to quickly prepare notes, quizzes, and presentations tailored to their students' needs.

5. Curriculum Developers and Administrators:

Professionals responsible for designing courses and academic programs.

Use case: EduPilot assists in generating exam papers, assignments, and structured learning materials for effective curriculum delivery.

5.3 Geographical Target Area:

- **Schools and Colleges:** Urban and suburban educational institutions where large numbers of students and teachers require organized study materials, quizzes, and assignments.
- **Online Learning Platforms:** Digital environments serving remote learners across different regions who need access to notes, exams, and interactive study resources.
- **Universities and Academic Centers:** Institutions with multiple departments and courses that can benefit from automated content generation and scheduling.

5.4 Configuration

EduPilot does not require complex or advanced configurations because it is designed as a single, unified platform for teachers and students. Users can install and run the application directly without additional setup, allowing immediate access to features such as notes generation, quizzes, assignments, and presentation topics.

5.5 Deployment places

5.5.1 Vercel Deployment(Recommended):

Steps:

1. Create a Vercel Account:

- Sign up at <https://vercel.com> using GitHub, GitLab, or Bitbucket.

2. Connect Your Repository:

- Link your Next.js project repository to Vercel.

3. Configure Build Settings:

- Vercel automatically detects Next.js. You can set environment variables if needed.

4. Deploy the App:

- Click “Deploy,” and Vercel will build and host your app.

- You will get a live URL (e.g., <https://edu-pilot.vercel.app>).

5. Custom Domain (Optional):

- You can add a custom domain like www.edupilot.com and configure DNS settings in Vercel.

5.5.2 Netlify Deployment:

Steps:

1. Create a Netlify Account:

- Sign up at <https://www.netlify.com>.

2. Connect Repository:

- Link your GitHub, GitLab, or Bitbucket repository.

3. Set Build Command:

- Build command: `npm run build` or `yarn build`
- Publish directory: `.next` or `out` (if using `next export`)

4. Deploy:

- Click deploy and Netlify will provide a live URL.

5. Custom Domain (Optional):

- Add your own domain and configure DNS in Netlify.

5.5.3 Cloud Hosting (AWS, DigitalOcean, or Render)

Steps:

1. Provision a Server:

- Use AWS EC2, DigitalOcean Droplet, or Render instance.

2. Install Node.js and NPM/Yarn:

- Ensure the server has Node.js environment for running the Next.js app.

3. Upload Project:

- Clone your repository or upload project files to the server.

4. Build and Run App:

- Run `npm install` → `npm run build` → `npm start`

5. Configure Reverse Proxy (Optional):

- Use Nginx or Apache to serve the app on your domain.

5.6 Application Cost

1. Development Tools

- Next.js, Node.js, and Firebase – Free for basic usage
- Visual Studio Code – Free

2. Hosting and Deployment

- Vercel – Free tier; Paid plans from \$20/month
- Netlify – Free tier; Paid plans from \$19/month
- Cloud Hosting (AWS/DigitalOcean/Render) – \$5–\$15/month (small server)

3. Database Costs

- Firebase Realtime Database/Firestore – Free tier available; Paid plans vary with usage

4. Domain and SSL

- Custom Domain – \$10–\$15/year
- SSL Certificate – Free via Vercel/Netlify; Small cost from providers if purchased separately

5. Maintenance and Upgrades

- Regular updates, bug fixes, and feature enhancements may require additional developer time.

5.7 Advertisement

EduPilot will be promoted through social media platforms, YouTube, educational blogs, and relevant websites. The primary focus will be on **WhatsApp groups, Telegram channels, and student communities** in schools, colleges, and universities. Students and teachers will be encouraged to share the app link with peers, helping to expand user adoption organically.

5.8 Hardware Integration and Fabrication

EduPilot does not require any hardware integration, as it is entirely a **web-based software platform**. No specialized hardware components are needed, simplifying deployment and usage for all users.

5.9 User Training

Training sessions will be provided to ensure users, including teachers and students, understand how to navigate the EduPilot platform effectively. Training will cover **accessing generated notes, quizzes, assignments, and presentation topics**, along with instructions on submitting inputs and tracking schedules. Since EduPilot is a real-time system, potential issues and troubleshooting methods will also be discussed during training.

5.10 Future Enhancements:

EduPilot, in its current version, automates course content generation, quiz management, and exam scheduling. However, several enhancements can be introduced in future to make the system more intelligent, accessible, and user-friendly:

1. Voice Integration

- Add support for **speech-to-text** and **text-to-speech** features.
- Teachers can dictate course contents, and students can listen to notes and quizzes.
- Improves accessibility for visually impaired users.

2. Multilingual Support

- Extend application support for multiple languages such as **Urdu, Arabic, Chinese, and French** in addition to English.
- This will make EduPilot usable across international institutions and improve inclusivity.

3. Custom Paper Branding

- Allow teachers/universities to **embed their logo, name, semester, subject, and teacher information** in automatically generated quizzes, assignments, and exam papers.
- Provides a professional and institution-specific look.

4. AI-Powered Student Feedback

- Add a feature where EduPilot analyzes quiz and assignment results to give **personalized feedback** and study suggestions to students.

5. Smart Notifications

- Integration with **WhatsApp, Telegram and Email** for automatic reminders of quiz deadlines, assignments, and exams.

6. Collaboration Features

- Enable **teacher-to-teacher sharing** of course materials, assignments, and quizzes.
- A repository or library system where teachers can upload and reuse generated content.

7. Offline Mode

- Provide offline functionality where users can access previously generated notes, assignments, and quizzes without internet.

8. **Cloud Integration & Backup**

- Sync generated course content to cloud storage (Google Drive, OneDrive) for backup and accessibility across devices.

9. **Gamification for Students**

- Add **leaderboards, badges, and rewards** for students who perform well in quizzes and assignments.
- Keeps learners motivated and engaged.

10. **Mobile App Enhancements**

- Integration of **push notifications, dark mode, and AI chatbot support** inside the Flutter app to improve student/teacher interaction.

LITERATURE CITED

- Al-Samarraie, H., Teng, B. K., Alzahrani, A. I., & Alalwan, N. (2018). E-learning continuance satisfaction in higher education: A unified perspective from instructors and students. *Studies in Higher Education*, 43(11), 2003–2019.
- Zawacki-Richter, O., Marín, V. I., Bond, M., & Gouverneur, F. (2019). Systematic review of research on artificial intelligence applications in higher education – Where are the educators? *International Journal of Educational Technology in Higher Education*, 16(1), 39–55.
- Chen, C. M., & Li, Y. L. (2010). Personalized context-aware ubiquitous learning system for supporting effective English vocabulary learning. *Interactive Learning Environments*, 18(4), 341–364
- Anderson, T., & Dron, J. (2011). *Teaching crowds: Learning and social media*. Athabasca University Press.
- Mayer, R. E. (2014). *The Cambridge handbook of multimedia learning* (2nd ed.). Cambridge University Press.
- OpenAI. (n.d.). OpenAI API documentation. Retrieved from <https://platform.openai.com/docs>
- LangChain. (n.d.). LangChain framework documentation. Retrieved from <https://docs.langchain.com>
- Firebase. (n.d.). Firebase documentation. Retrieved from <https://firebase.google.com/docs>
- Stanford University. (2020). SQuAD: Stanford Question Answering Dataset. Retrieved from <https://rajpurkar.github.io/SQuAD-explorer/>
- Kaggle. (2021). Educational question-answering and quiz datasets. Retrieved from <https://www.kaggle.com/datasets>

SHORT BIOGRAPHY OF THE SCHOLAR

Muhammad Nasir is a final-year Computer Science student at The University of Swabi, known for his strong interest in Artificial Intelligence, software engineering, and real-world automation. His academic journey reflects steady progress from foundational programming to designing intelligent, production-level systems. This growth led him to develop his final-year project, **Education Pilot**, an AI-powered platform that automates the creation of educational material for teachers and institutions.

Education Pilot is capable of generating topic explanations, assignments, quizzes, presentations, mid-term and final examination papers, and complete course timelines from simple uploaded course content. What makes this project notable is that **every component—design, development, AI integration, testing, and optimization—was completed independently by Muhammad Nasir** without any external development support.

He designed the system architecture, built the backend, implemented OCR to extract text from images and PDFs, integrated AI models for content generation, and created the frontend to ensure a clean and user-friendly experience. His ability to plan, design, and execute the entire project shows a solid understanding of full-stack development, prompt engineering, and modern software practices.

Nasir focused especially on the accuracy and reliability of AI-generated material. He thoroughly tested each module, refined the difficulty levels, and ensured that the system produced academically sound content. His work demonstrates a practical mindset aimed at solving real problems faced by teachers—especially the time required to prepare notes, quizzes, and exam papers manually.

Education Pilot stands as evidence of his technical discipline, independence, and problem-solving abilities. Instead of choosing a simple project, he built a complete automation system that can be extended for schools, colleges, and training platforms.

Looking ahead, Muhammad Nasir plans to pursue advanced studies in Artificial Intelligence and Software Engineering. With the successful completion of Education Pilot, he has shown the capability to turn complex ideas into functional solutions, positioning himself as a promising developer ready to contribute to future educational technology.